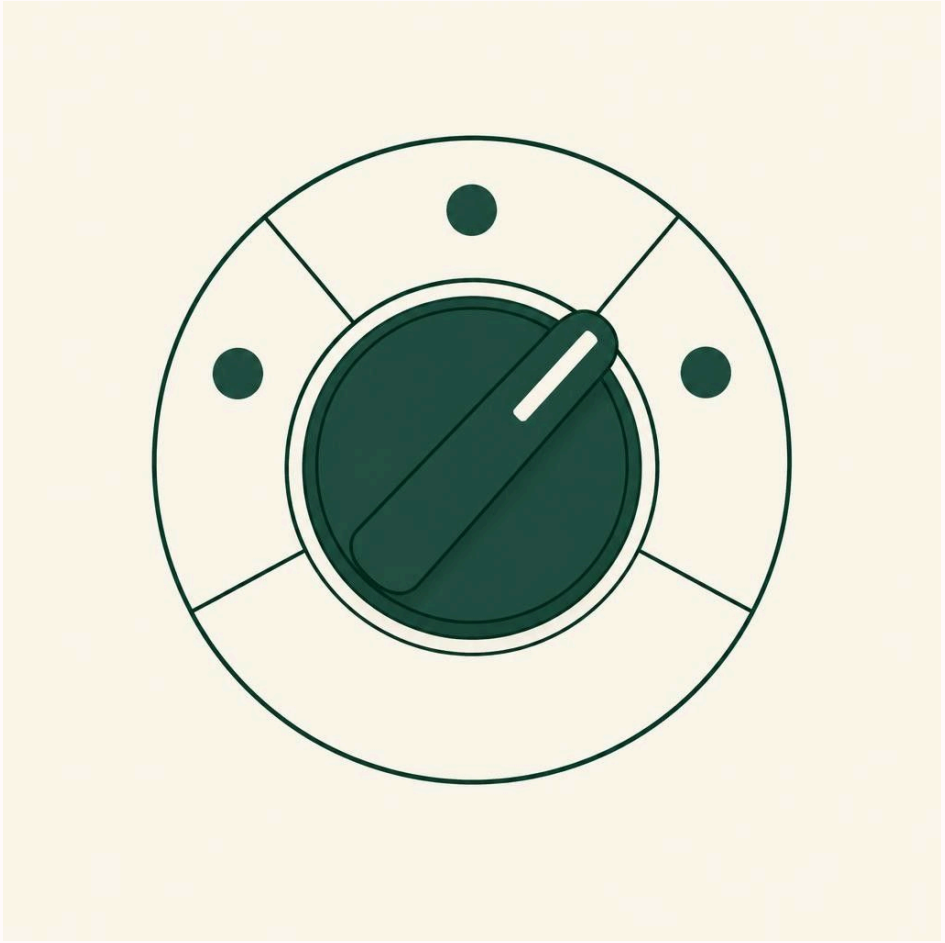
Enum in C

Dichiarazione e valori, enum e switch, enum nelle struct

LINGUAGGIO C | STRUTTURE E UNIONI | ENUM



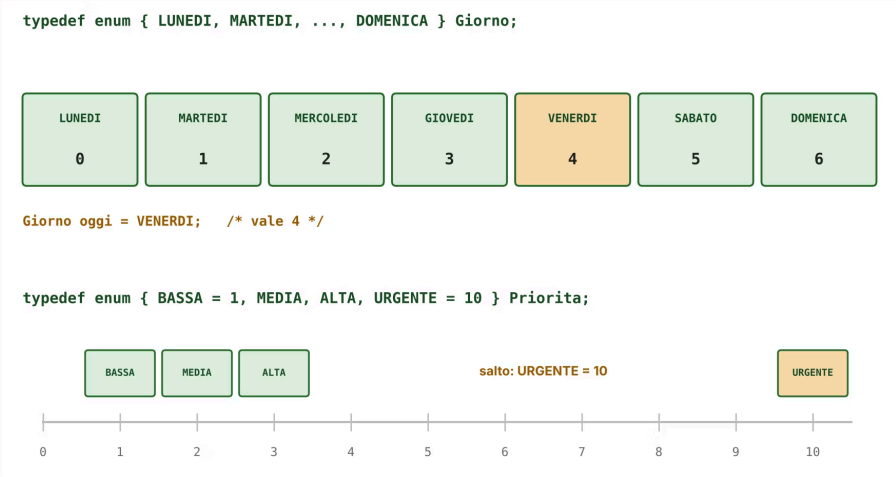
Un Tipo con Costanti con Nome

Un enum elenca i valori possibili di un tipo, dando un nome a ognuno. Le costanti sono interi: la prima vale 0, ognuna vale uno in più della precedente.

```
typedef enum {
    LUNEDI, MARTEDI, MERCOLEDI, GIOVEDI,
    VENERDI, SABATO, DOMENICA
} Giorno;

Giorno oggi = VENERDI;        /* vale 4 */

if (oggi == SABATO || oggi == DOMENICA) {
    printf("fine settimana\n");
}
```



> Convenzione del corso: il tipo ha l'iniziale maiuscola come le struct, le costanti sono tutte maiuscole.

#define o enum?

```
#define LUNEDI   0
#define MARTEDI  1
#define MERCOLEDI 2
...
int oggi = 4;            /* quale giorno? */

typedef enum { LUNEDI, MARTEDI, ... } Giorno;
Giorno oggi = VENERDI;
void stampa_orario(Giorno g);

oggi = 42;               /* compila lo stesso */
```

#define	enum
costanti sparse	un tipo con un nome
valori scritti a mano	valori assegnati dal compilatore
il debugger mostra 4	il debugger mostra VENERDI
nessun legame nei prototipi	il prototipo dice quali valori si aspetta

> Un enum non è un controllo: un numero letto da tastiera va verificato prima di assegnarlo a una variabile enum.

Valori Espliciti

```
typedef enum {
    GENNAIO = 1, FEBBRAIO, MARZO, ..., DICEMBRE
} Mese;                          /* da 1 a 12 */

typedef enum {
    BASSA = 1, MEDIA, ALTA, URGENTE = 10
} Priorita;                      /* 1, 2, 3, 10 */

typedef enum {
    OK = 200, NON_TROVATO = 404,
    ERRORE_SERVER = 500, SUCCESSO = OK
} Risposta;
```

Con = si sceglie il valore di una costante; le successive continuano a contare da lì. Due costanti possono avere lo stesso valore (SUCCESSO è un sinonimo di OK).

> Con i valori scelti a mano il numero ha un significato anche fuori dal programma: GENNAIO = 1 coincide con il mese scritto in una data, NON_TROVATO = 404 con il codice ricevuto dal browser.

Enum come Indice

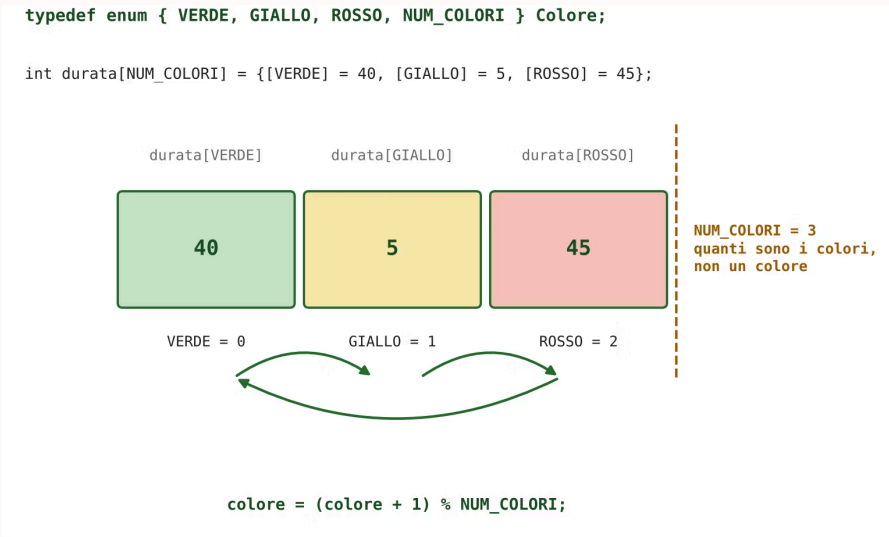
Le costanti partono da 0 e salgono di uno: sono gli indici di un array. Una costante finale conta le altre.

```
typedef enum { VERDE, GIALLO, ROSSO, NUM_COLORI } Colore;

int durata[NUM_COLORI] = {[VERDE] = 40, [GIALLO] = 5, [ROSSO] = 45};
const char *nomi[NUM_COLORI] = {[VERDE] = "VERDE",
                                [GIALLO] = "GIALLO",
                                [ROSSO] = "ROSSO"};

printf("%s per %d s\n", nomi[colore], durata[colore]);
colore = (colore + 1) % NUM_COLORI;
```

Aggiungendo un colore prima di NUM_COLORI, gli array si ridimensionano da soli. Con [VERDE] = ... ogni valore sta accanto alla sua costante.



switch su un Enum

```
double area(Forma f, double misura)
{
    double risultato = 0;

    switch (f) {
    case CERCHIO:
        risultato = PI_GRECO * misura * misura;
        break;
    case QUADRATO:
        risultato = misura * misura;
        break;
    case TRIANGOLO:
        risultato = sqrt(3.0) / 4 * misura * misura;
        break;
    }
    return risultato;
}
```

Un enum elenca tutti i valori possibili, quindi lo switch ha un case per ognuno. Con -Wall il compilatore controlla:

```
warning: enumeration value 'TRIANGOLO'
        not handled in switch
```

> Sugli enum niente default: quando si aggiunge una costante, il compilatore indica tutti gli switch da aggiornare. La costante NUM_... ha il suo case vuoto.

Funzioni con Enum e Conversione da Testo

```
typedef enum {
    ITALIANO, MATEMATICA, INGLESE, INFORMATICA,
    NUM_MATERIE,
    MATERIA_NON_VALIDA
} Materia;

Materia da_testo(const char *parola)
{
    Materia trovata = MATERIA_NON_VALIDA;
    Materia m = ITALIANO;

    while (m < NUM_MATERIE && trovata == MATERIA_NON_VALIDA) {
        if (strcmp(a_testo(m), parola) == 0) {
            trovata = m;
        }
        m++;
    }
    return trovata;
}
```

Funzione	Al posto di
Esito valuta(double media, int insuff.)	int che vale 0, 1 o 2
const char *a_testo(Materia m)	uno switch ripetuto
Materia da_testo(const char *parola)	strcmp sparsi nel codice

> Il valore per le parole sconosciute sta dopo NUM_MATERIE: non diventa mai un indice valido della tabella dei nomi.

Un Campo Enum

```
typedef enum {
    DISPONIBILE, IN_PRESTITO, IN_RIPARAZIONE, NUM_STATI
} Stato;

typedef struct {
    char titolo[28];
    Stato stato;
} Libro;

for (i = 0; i < n; i++) {
    conteggi[libri[i].stato]++;
}
```

Lo stato di un oggetto è spesso uno tra pochi valori: è il caso tipico di un campo enum. Lo stato è anche un indice per contare i libri di ogni stato in un solo ciclo.

> Due campi int (prestato, in_riparazione) possono contraddirsi; un solo campo enum ammette un solo stato alla volta.

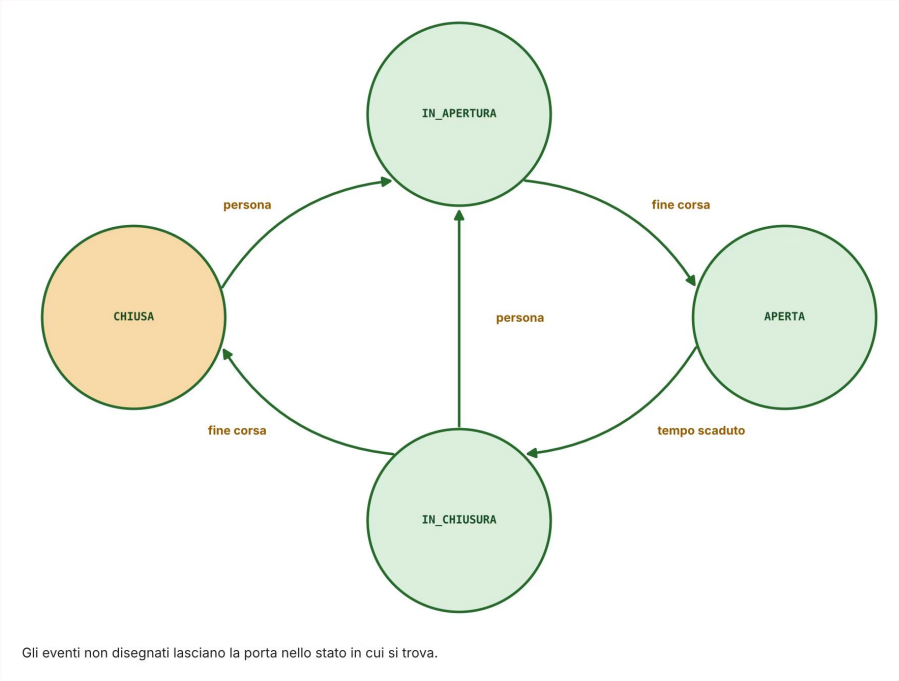
Macchine a Stati

Una macchina a stati è in un solo stato alla volta; ogni evento la fa passare in un nuovo stato o la lascia dov'è. Stati ed eventi sono due enum.

```
typedef enum {
    CHIUSA, IN_APERTURA, APERTA, IN_CHIUSURA, NUM_STATI
} Stato;

typedef enum {
    PERSONA, FINE_CORSA, TEMPO_SCADUTO, NUM_EVENTI
} Evento;
```

Le regole si scrivono con una tabella delle transizioni, oppure con uno switch sullo stato quando ogni passo fa anche altro: l'ascensore si sposta di un piano, l'automa conta una parola.



Gli eventi non disegnati lasciano la porta nello stato in cui si trova.

La Tabella delle Transizioni

La riga è lo stato attuale, la colonna l'evento, la cella il nuovo stato. Tutta la logica è nei dati: il ciclo che fa funzionare la porta è una riga sola.

```
Stato transizioni[NUM_STATI][NUM_EVENTI] = {
    [CHIUSA] =      {IN_APERTURA, CHIUSA, CHIUSA},
    [IN_APERTURA] = {IN_APERTURA, APERTA, IN_APERTURA},
    [APERTA] =      {APERTA,      APERTA, IN_CHIUSURA},
    [IN_CHIUSURA] = {IN_APERTURA, CHIUSA, IN_CHIUSURA}
};

stato = transizioni[stato][evento];
```

	PERSONA	FINE_CORSA	TEMPO_SCADUTO
[CHIUSA]	**IN_APERTURA**	CHIUSA	CHIUSA
[IN_APERTURA]	IN_APERTURA	**APERTA**	IN_APERTURA
[APERTA]	APERTA	APERTA	**IN_CHIUSURA**
[IN_CHIUSURA]	**IN_APERTURA**	**CHIUSA**	IN_CHIUSURA

```
stato = transizioni[APERTA][TEMPO_SCADUTO];   /* IN_CHIUSURA */
```

Le celle bianche lasciano lo stato invariato.

> Per cambiare il comportamento si cambia una cella, non il codice. Con la stessa idea, nei ticket una tabella permesso[da][a] dice quali cambi di stato sono ammessi.

Errori Comuni da Evitare

1

Aspettarsi che un enum controlli i valori: oggi = 42 compila.

2

Indicizzare una tabella con un enum che parte da 1 (DICEMBRE vale 12).

3

Scrivere a mano la dimensione degli array invece di NUM_....

4

Usare la costante di conteggio come se fosse un valore.

5

Stampare il nome con %s invece di nomi[valore].

6

Mettere default in uno switch su enum e perdere l'avviso del compilatore.

7

Usare MATERIA_NON_VALIDA come indice di un array.

8

Assegnare a un campo una costante di un altro enum (-Wenum-conversion).

Riepilogo Concetti

Situazione	Tecnica	Note
Valori con un nome	`typedef enum { A, B, C } Tipo;`	da 0, di uno in uno
Valori scelti	`GENNAIO = 1, URGENTE = 10`	le successive continuano
Contare i valori	costante finale NUM_...	dimensione degli array
Nomi da stampare	nomi[valore], [VERDE] = "..."	una tabella sola
Valore successivo	(v + 1) % NUM_...	ciclo chiuso
Scegliere il caso	switch con tutti i case	niente default
Risultato di una funzione	Esito valuta(...)	niente numeri magici
Da testo a enum	ricerca nella tabella dei nomi	valore NON_VALIDO dopo NUM_
Stato di un oggetto	campo enum nella struct	un solo stato alla volta
Regole tra stati	transizioni[stato][evento]	la logica nei dati

SEZIONE 1

Esempi Pratici: Dichiarazione e Valori

Concetti base — tre file su costanti, valori e indici.

1 – 010-costanti-con-nome.c

Problema

i giorni della settimana e il fine settimana.

Tecnica

Giorno, valori da 0, confronti e sizeof.

2 – 020-valori-espliciti.c

Problema

mesi, priorità dei guasti e codici del server web.

Tecnica

valori con =, salti e sinonimi.

3 – 030-enum-come-indice.c

Problema

le ore di lezione della 3B.

Tecnica

NUM_GIORNI, array di ore e tabella dei nomi.

SEZIONE 1

Esempi Pratici: Problemi su Dichiarazione e Valori

040 – Semaforo

Il colore dopo t secondi con (colore + 1) % NUM_COLORI.

050 – Giorni di lezione

Ore per giorno, giorno più lungo e giorni liberi.

060 – Taglie in magazzino

Scorte di magliette per taglia, sigla cercata nella tabella.

070 – Mesi e stagioni

Mese da GENNAIO = 1: giorni del mese e stagione.

080 – Livelli di log

Livelli ordinati per gravità: filtro con >= e conteggi.

SEZIONE 2

Esempi Pratici: Enum e Switch

Concetti base — tre file su switch, funzioni e conversioni.

1 – 010-switch-su-enum.c

Problema

l'area delle figure per il compito di geometria.

Tecnica

switch completo e avviso di -Wswitch.

2 – 020-funzioni-con-enum.c

Problema

lo scrutinio finale.

Tecnica

Esito valuta(...) e descrizione(e).

3 – 030-da-testo-a-enum.c

Problema

le ore svolte per materia.

Tecnica

da_testo e a_testo con una tabella sola.

SEZIONE 2

Esempi Pratici: Problemi su Enum e Switch

040 – Distributore di bevande

Prezzo con uno switch, Esito dell'erogazione.

050 – Giudizio sintetico

Dal voto al giudizio e istogramma della classe.

060 – Calcolatrice a parole

Operazione scritta come parola e switch del calcolo.

070 – Direzioni del robot

Direzioni in ordine orario: rotazioni con il modulo.

080 – Prenotazione dell'aula magna

Un Esito per ogni motivo di errore e messaggio con uno switch.

SEZIONE 3

Esempi Pratici: Enum nelle Struct

Concetti base — tre file su campi, stati e tabelle.

1 – 010-campo-enum.c

Problema

la biblioteca di classe.

Tecnica

campo Stato, conteggi e filtri per stato.

2 – 020-macchina-a-stati.c

Problema

la porta automatica dell'ingresso.

Tecnica

tabella delle transizioni e sequenza di eventi.

3 – 030-tabella-indicizzata-da-enum.c

Problema

i biglietti dello spettacolo di fine anno.

Tecnica

array di Tariffa indicizzato dalla categoria.

SEZIONE 3

Esempi Pratici: Problemi su Enum nelle Struct

040 – Ticket di assistenza

Priorità e stato, cambi permessi da una tabella.

050 – Ascensore

Macchina a stati con uno switch: porte, salita e discesa.

060 – Tessere della mensa

Fascia ISEE come campo enum e tabella dei prezzi.

070 – Gioco del tris

Griglia di Casella, turni e Risultato.

080 – Analisi del testo

Automa che conta parole, numeri e punteggiatura.